

3. Multithreaded Programming

Dr. Arun Kumar
Dept. of CSE, NIT Rourkela

Threads

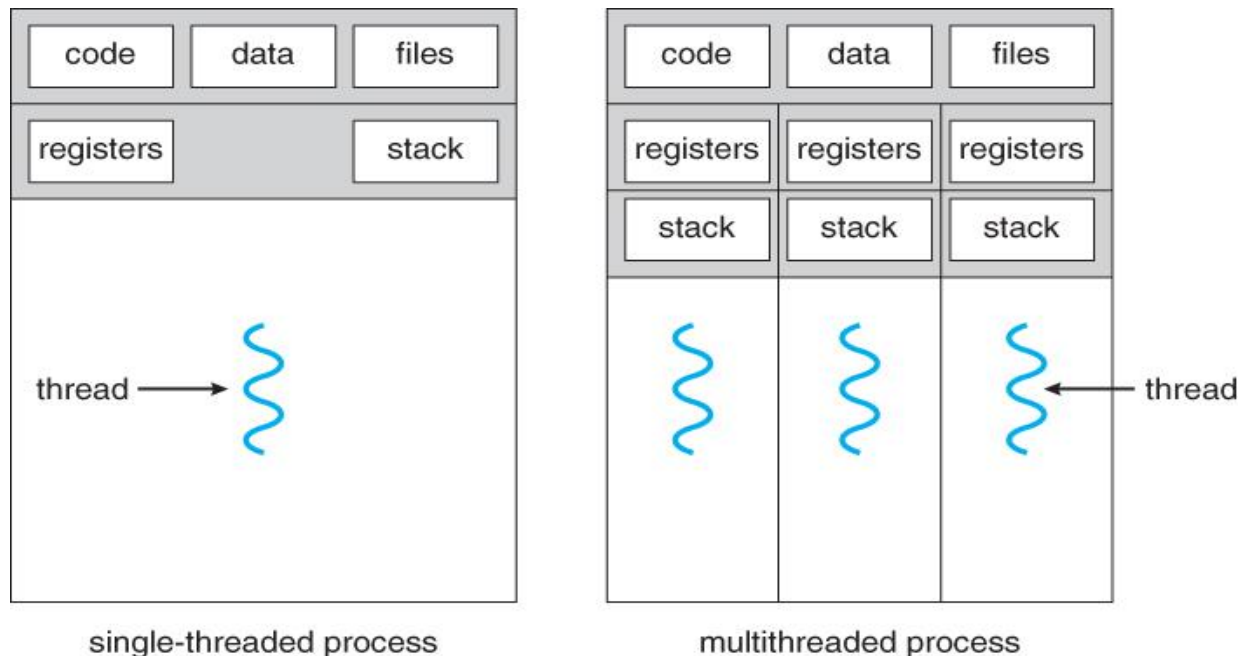
- ▶ A traditional process has a *single thread* of control.
 - ▶ There is one *program counter*, and *one sequence of instructions* that can be carried out at any given time.
- ▶ Is it possible to have **only one processor and perform multiple tasks?**
 - ▶ Yes, **Multi programming** etc.
 - ▶ Program might have **multiple processes and each process can have multiple threads.**

Threads

- ▶ A thread is a *basic unit of CPU utilization*.
- ▶ It comprises a *thread ID*, a *program counter*, a *register set* and a *stack*. 
- ▶ A thread *shares* with other threads belonging to the same process its *code section*, *data section*, and other OS resources, such as *open files and signals*.

Threads

- ▶ As shown in Figure, **multi-threaded applications have multiple threads within a single process**, each having their own *program counter*, *stack* and set of *registers*, but sharing common *code*, *data*, and certain structures such as *open files*.



Threads

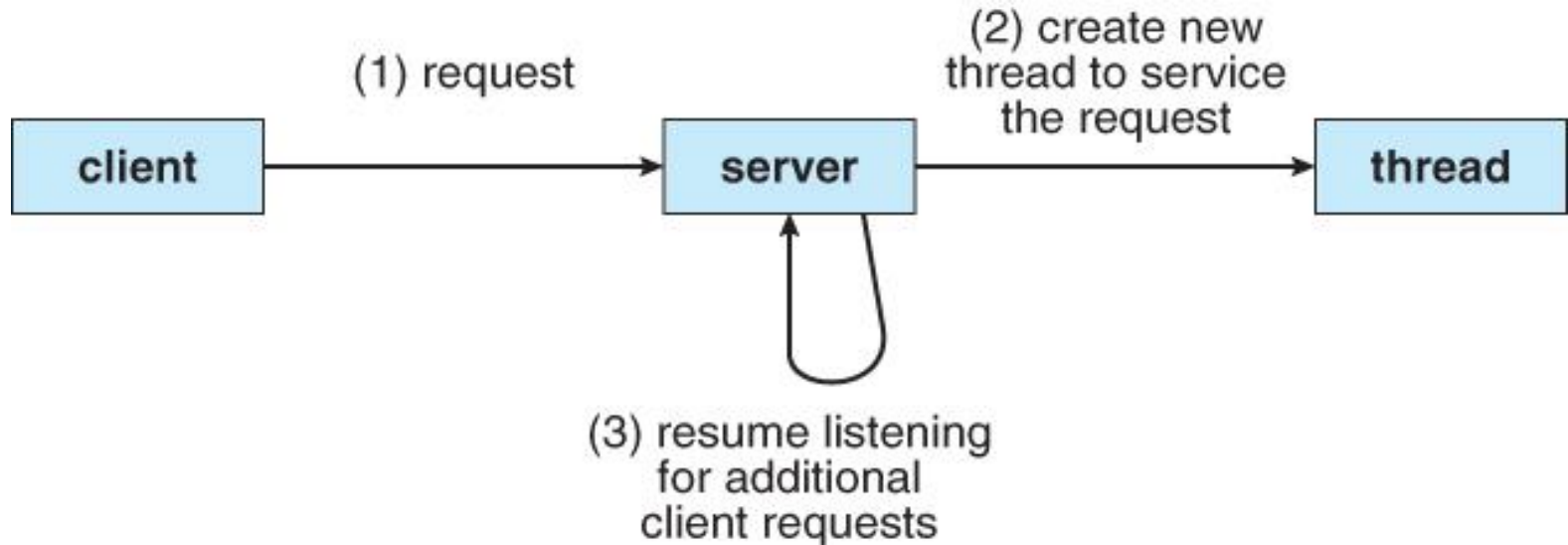
- ▶ **Software:** *ProcessThreadsView* – **chrome.exe**
 - ▶ Multiple processes and each process with multiple threads
- ▶ Chrome is able to *display a page* as well as *download a file* at a same time as it supports *multiple threads*.
 - ▶ *Separate threads for display* the image and *downloading* the files are running.

Example 2

- ▶ Threads are very useful in modern programming whenever a process has *multiple tasks* to perform *independently* of the others.
- ▶ This is particularly true when *one of the tasks may block*, and it is desired to allow the *other tasks* to proceed *without blocking*.
- ▶ For example in a *word processor*, a *background thread* may *check spelling* and *grammar* while a *foreground thread* processes *user input* (keystrokes), while yet a *third thread loads images from the hard drive*, and a fourth does *periodic automatic backups* of the file being edited.

Example 3

- ▶ Another example is a **web server** - Multiple threads allow for multiple requests to be satisfied simultaneously, without having to service requests sequentially or to fork off separate processes for every incoming request.



Benefits

- ▶ **Responsiveness:** Multithreading an interactive application may allow a program to continue running even if a part of it is blocked or is performing a lengthy operation, thereby increasing *responsiveness* to the user.
 - ▶ One thread may provide rapid response while other threads are blocked or slowed down doing *intensive calculations*.
- ▶ **Resource sharing:** By default threads share common code, data, and other resources, which allows multiple tasks to be performed simultaneously in a single address space.

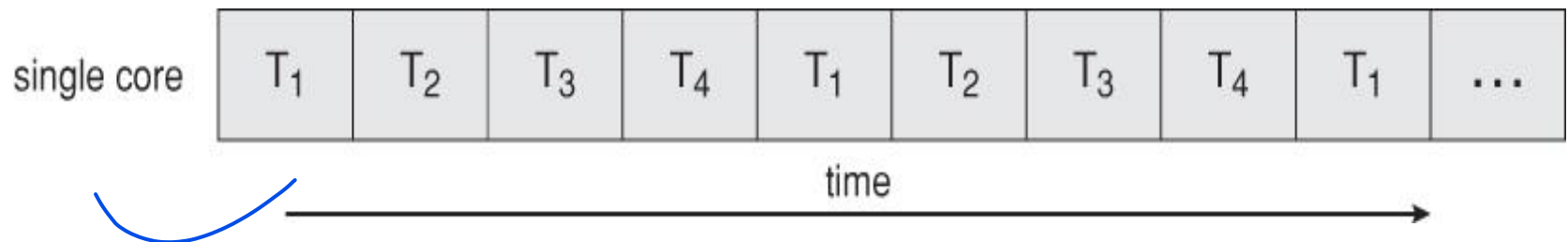
Benefits

RDES
X

- ▶ **Economy:** Allocating memory and resources for *process creation* is *costly*. As the threads shares the resources of the process which they belong, it is *more economical to create and context-switch threads*.
- ▶ Creating and managing threads (context switches between them) is much faster than performing the same tasks for processes.
- ▶ In Solaris, **creating a process is about 30 times slower** than creating a thread, and **context switching is about five times slower**.
- ▶ **Scalability:** A single threaded process can only run on one CPU, no matter how many may be available, whereas the execution of a *multi-threaded application* may be split amongst available processors.

Multicore Programming

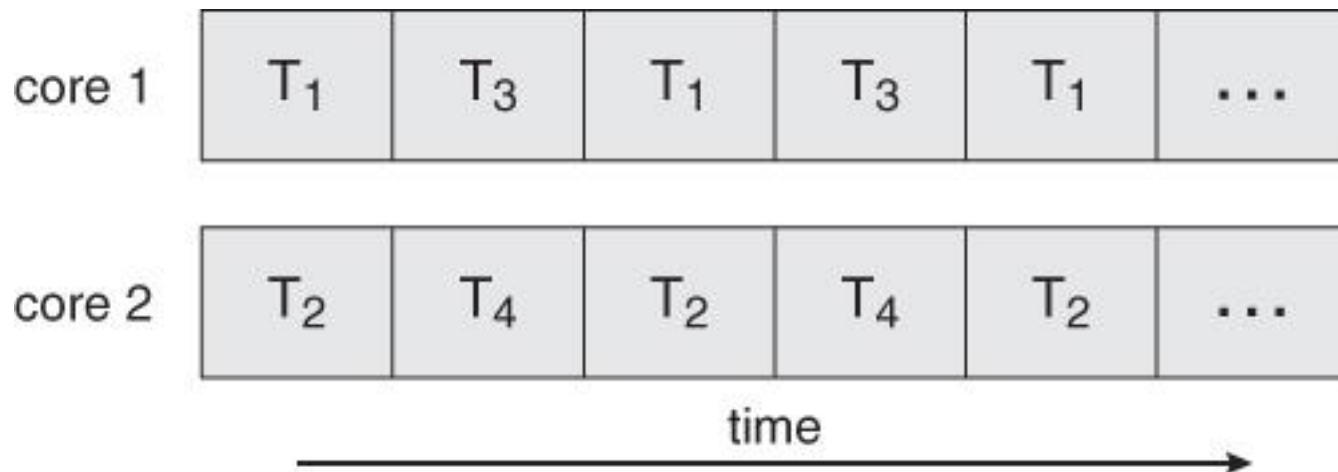
- ▶ A recent trend in computer architecture is to produce chips with multiple **cores**, or CPUs on a single chip.
- ▶ A multi-threaded application running on a traditional **single-core** chip would have to interleave the threads as shown in Figure.



Concurrent execution on a single-core system.

Multicore Programming

- ▶ On a *multi-core* chip, however, the threads could be spread across the available cores, allowing *true parallel processing*.



Parallel execution on a multicore system

Multicore Programming

- ▶ For operating systems, **multi-core** chips require new *scheduling algorithms* to make better use of the multiple cores available.
- ▶ As *multi-threading* becomes more *pervasive* and more *important (thousands instead of tens of threads)*, CPUs have been developed to support **more simultaneous threads per core** in hardware.

Programming Challenges

For application programmers, there are **five areas** where **multi-core** chips present new challenges:

- ▶ **Identifying tasks** - Examining **applications** to find *activities* that can be performed **concurrently**.
 - ▶ Tasks which are **independed of one another** and thus can run on **individual cores**.
- ▶ **Balance** - Finding *tasks* to run *concurrently* that provide *equal value*. I.e. don't waste a thread on trivial tasks.
 - ▶ Using a *separate execution core* to run an unnecessary task may not be *worth the cost*.

Programming Challenges

- ▶ **Data splitting** – As the applications are divided into separate tasks, the *data accessed and manipulated* by the task must be divided to run on separate cores.
 - ▶ To prevent the threads from interfering with one another.
- ▶ **Data dependency** - If one task is *dependent* upon the *results of another*, then the tasks need to be **synchronized** to assure access in the proper order.

Programming Challenges

- ▶ **Testing and debugging** – When a *program* is running in *parallel on multiple cores*, many *different execution paths* are possible.
 - ▶ *Testing and debugging* such *concurrent programs* is inherently more *difficult* than the testing and debugging the *single threaded applications*.
 - ▶ *The race conditions* become much more *complex and difficult* to identify.

Types of Parallelism

In general, there are two types of parallelism:

► **Data parallelism** *divides the data up amongst multiple cores, and performs the **same task on each subset of the data.***

► Example: dividing a *large image up into pieces* and performing the *same digital image processing* on each piece on *different cores*.

► *Summing* the contents of *an array of Size N*. On *a single core* all the elements needs to be summed up $[0] \dots [N-1]$. On *a dual core system*, *thread A* running on *core 0*, could sum the elements $[0] \dots [N/2-1]$ while *thread B*, running on *core 1*, could sum the elements $[N/2] \dots [N-1]$.

Types of Parallelism

- ▶ **Task parallelism** involves distributing *not data but task* (threads) across multiple cores.
 - ▶ Each thread is performing a unique operation.
 - ▶ Different threads may be operating on the same data, or they may be operating on different data.
 - ▶ *Example:* Consider the **array example** earlier, in contrast to that situation, *task parallelism might involve two threads, each performing a unique statistical operation on the array of the elements (One core performing addition and other one multiplication etc.)*.
 - ▶ *The thread again are operating in parallel on separate computing cores, but each is performing a unique operation.*

Types of Parallelism

- ▶ Fundamentally, then, *data parallelism* involves the *distribution of data across multiple cores* and *task parallelism* on the *distribution of tasks across multiple cores*.
- ▶ *In practice no program is ever divided up solely by one or the other of these, but instead by some sort of **hybrid combination**.*

Threads Vs Processes

Comparison Basis	Process	Thread
Definition	A process is a program under execution i.e an active program .	A thread is a lightweight process that can be managed independently by a scheduler .
Context switching time	Processes require more time for context switching as they are more heavy.	Threads require less time for context switching as they are lighter than processes.
Memory Sharing	Processes are totally independent and don't share memory.	A thread may share some memory with its peer threads.
Communication	Communication between processes requires more time than between threads.	Communication between threads requires less time than between processes .

Threads Vs Processes

Comparison Basis	Process	Thread
<i>Blocked</i>	If a process gets blocked, remaining processes can continue execution.	If a user level thread gets blocked, all of its peer threads also get blocked.
<i>Resource Consumption</i>	Processes require more resources than threads.	Threads generally need less resources than processes.
<i>Dependency</i>	Individual processes are independent of each other.	Threads are parts of a process and so are dependent.
<i>Data and Code sharing</i>	Processes have independent data and code segments.	A thread shares the data segment, code segment, files etc. with its peer threads.

Threads Vs Processes

Comparison Basis	Process	Thread
<i>Treatment by OS</i>	All the different processes are treated separately by the operating system.	All user level peer threads are treated as a single task by the operating system.
<i>Time for creation</i>	Processes require more time for creation.	Threads require less time for creation.
<i>Time for termination</i>	Processes require more time for termination.	Threads require less time for termination.

Multithreading Models

There are *two types of threads* to be managed in a modern system: *User threads* and *kernel threads*:

- ▶ ***User threads*** are supported *above the kernel*, without kernel support. These are the *threads that application programmers would put into their programs*.
- ▶ ***Kernel threads*** are supported *within the kernel* of the OS itself. All *modern OSs support* kernel level threads, allowing the kernel to perform *multiple simultaneous tasks* and/or to service *multiple kernel system calls* simultaneously.
- ▶ To *communicate*, ultimately,  a relationship must exist between the *user threads* and *kernel threads*.

User Level thread and Kernel Level thread

| L

User level thread	Kernel level thread
User thread are managed by user level libraries. Pthread.h or Pthread_create	kernel threads are implemented by OS. (System calls)
OS doesn't recognized user level threads.	Kernel threads are recognized by OS.
Implementation of User threads is easy.	Implementation of Kernel thread is complicated.
Context switch time is less. (OS is not involved)	Context switch time is more.

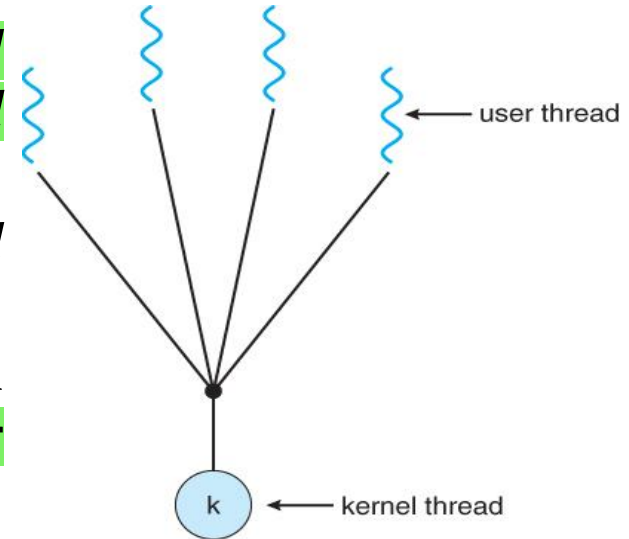
User Level thread and Kernel Level thread

User level thread	Kernel level thread
Context switch requires no hardware support.	Hardware support is needed.
If one user level thread perform blocking operation then entire process will be blocked.	If one kernel thread perform blocking operation then another thread can continue execution.
User level threads are designed as dependent threads.	Kernel level threads are designed as independent threads.
Example: Java thread, POSIX threads.	Example: Window Solaris.

Many-To-One Model

MBK

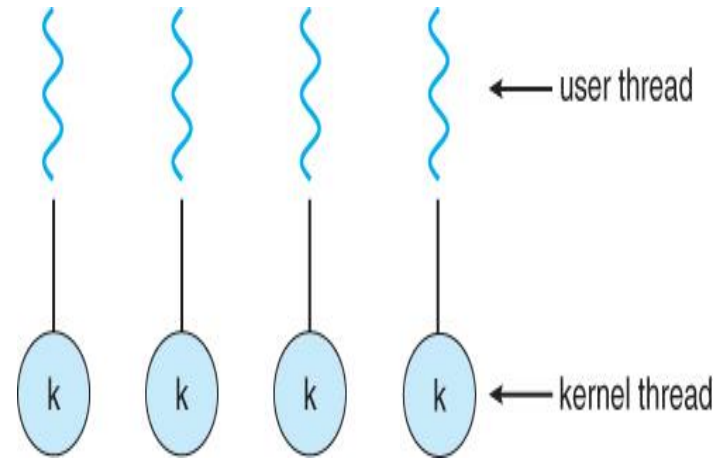
- ▶ In the many-to-one model, *many user-level threads are all mapped onto a single kernel thread.*
- ▶ Thread management is handled by the *thread library* in user space, which is very *efficient*.
- ▶ However, if a *blocking system call* is made, then the *entire process blocks*, even if the other user threads would otherwise be able to continue.
- ▶ *Because a single kernel thread can operate only on a single CPU, the many-to-one model does not allow individual processes to be split across multiple CPUs.*
- ▶ Green threads for **Solaris and GNU** Portable Threads implement the many-to-one model in the past, but few systems continue to do so today.



One-To-One Model

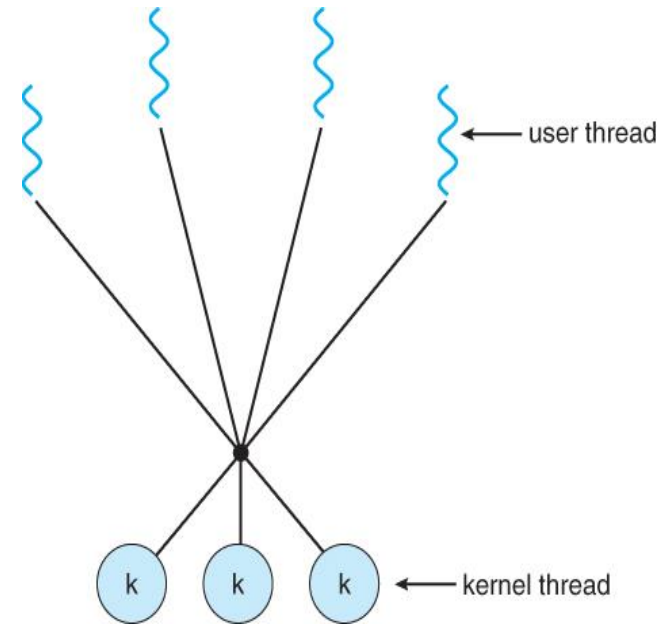
BOV

- ▶ The *one-to-one model* creates a **separate kernel thread** to handle *each user thread*.
- ▶ One-to-one model **overcomes** the problems listed above involving **blocking system calls** and the **splitting of processes across multiple CPUs**.
- ▶ However the **overhead** of managing the one-to-one model is **more significant**, involving more overhead and **slowing down the system**.
- ▶ Most implementations of this model place a **limit** on how many threads can be created.
- ▶ **Linux** and **Windows** from **95** to **XP** implement the one-to-one model for threads.



Many-To-Many Model

- ▶ The many-to-many model multiplexes *any number of user threads* onto **an equal or smaller number of kernel threads**, combining the best features of the one-to-one and many-to-one models.
- ▶ Users have **no restrictions** on the number of threads created.
- ▶ **Blocking kernel system calls do not block the entire process.**
- ▶ **Processes can be split across multiple processors.**
- ▶ Individual processes may be allocated **variable numbers of kernel threads**, depending on the *number of CPUs present* and other factors.



Hyper-Threading/ simultaneous multithreading or SMT HT

- ▶ *Hyper-threading* is **Intel's term** for what's also called *simultaneous multithreading* or *SMT* in the computer industry.
- ▶ *Hyper-threading* is a process by which a CPU divides up its **physical cores** into **virtual cores** that are treated as if they are actually physical cores by the operating system.
- ▶ These *virtual cores* are also called *threads*.

Hyper-Threading/ simultaneous multithreading or SMT

- ▶ Most of Intel's CPUs with *2 cores* use this process to create 4 threads or *4 virtual cores*.
- ▶ Intel CPUs with *4 cores* use *hyper-threading* to create more power in the form of ***8 virtual cores, or 8 threads***.

Hyper-Threading/ simultaneous multithreading or SMT

Advantages:

- ▶ **Multitasking:** If system supports Hyper-Threading then several processes can run simultaneously on the different threads of the CPU's core.
- ▶ **Resource Utilization:** Using Hyper-Threading resources can't be stay idle and utilized in great extent, because of several simultaneously running processes that can need resources. Because of this, the system's overall *efficiency and performance increases*.
- ▶ **Responsiveness:** Concurrent execution helps to decrease the response time for user.

Hyper-Threading/ simultaneous multithreading or SMT

Disadvantages:

- ▶ If many processes are running simultaneously because of Hyper-Threading, and these processes compete for limited resources, such as cache memory then *resources may get blocked*.
- ▶ Because of many running processes, resources are shared among them, which can lead to increase the contention and *delays in input/output of resources*.

Thank You

```
1.  for(i=0;i<100000;i++)
2.  {
3.      receive data from i/p device  //RcvData()
4.      send data on communication link  //SendonLink()
5.  }
```

- ▶ If communication link is not free then the process will halt in line#4. Even though i/p device is free, it can be used.
- ▶ In line#3, if user has not specified any input then communication link may be underutilized.
 - ▶ If there is no other process available, then CPU can't make a context switch even (at line#3)

Threads Vs Processes

Solution

- ▶ Consider RcvData() and SendonLink() to be two different entities
- ▶ RcvData(), on data availability, will put data in a queue
- ▶ SendonLink(), on link availability, will send data from the queue
- ▶ Whoever is free, can execute on CPU

RcvData() and SendonLink() entities can be implemented as processes or Threads

Threads Vs Processes

- ▶ Threads are the unit of execution in a process.
- ▶ A thread shares address space with it's parent.
- ▶ Per Thread items
 - ▶ Thread ID – Unique identifier
 - ▶ Program Counter – which instruction to execute next
 - ▶ Registers – for computation
 - ▶ Stack – contains the execution history
 - ▶ State – thread state

Threads Vs Processes

- ▶ Threads share the address space of the process that created it; processes have their own address space.
- ▶ Threads have direct access to the data segment of its process; processes have their own copy of the data segment of the parent process.
- ▶ Threads can directly communicate with other threads of its process; processes must use interprocess communication to communicate with sibling processes.
- ▶ Threads have almost no overhead; processes have considerable overhead.
- ▶ New threads are easily created; new processes require duplication of the parent process.

It is a light weight process and faster

Thread: Programmer's View

```
void fn1(int arg0, int arg1, ...) {...}

main() {
    ...
    tid = CreateThread(fn1, arg0, arg1, ...);
    ...
}
```

- At the point `CreateThread` is called, execution continues in parent thread in main function, and execution starts at `fn1` in the child thread, both in parallel

How Thread Can Help? – Example 1

- Consider the following code fragment

```
for(k = 0; k < n; k++)  
    a[k] = b[k] * c[k] + d[k] * e[k];
```

- Rewrite this code fragment as:

```
CreateThread(fn, 0, n/2);  
CreateThread(fn, n/2, n);  
  
fn(l, m) {  
    for(k = l; k < m; k++)  
        a[k] = b[k] * c[k] + d[k] * e[k];  
}
```

- What did we gain?

How Thread Can Help? – Example 2

► Consider a Web server

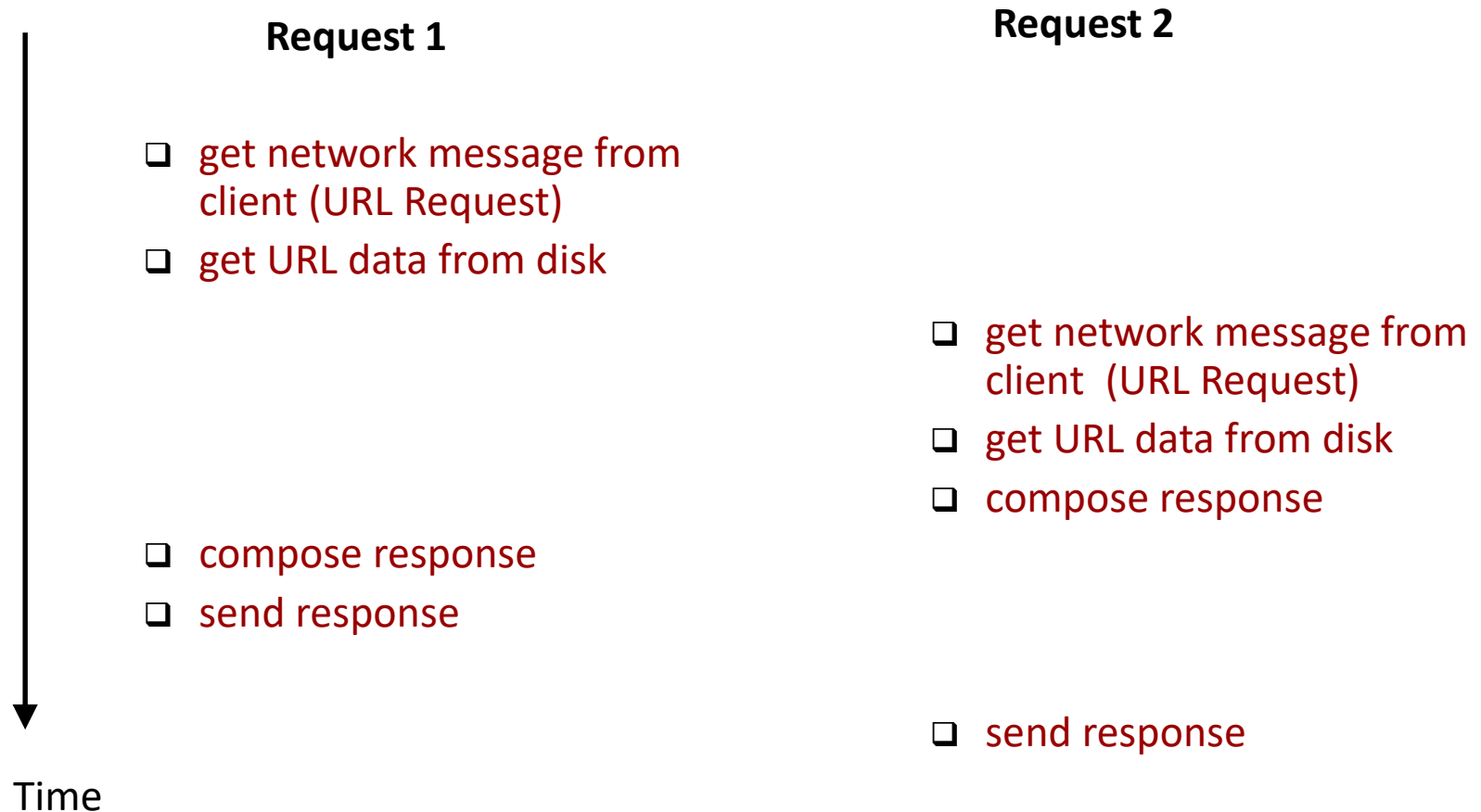
Create a number of threads, and for each thread do

- get network message from client(URL Request)
- get URL data from disk
- compose response
- send response

► What did we gain?

How Thread Can Help? – Example 2

Overlapping Requests (Concurrency)



System Call

- ▶ A request by an active process to the Kernel for a service
- ▶ Defines the interface between the user and the OS
- ▶ The process switches to Kernel mode from user mode, during a system call
- ▶ Preemptive vs. non-preemptive kernel
 - ▶ NonPreemptive: Linux 2.4 Kernel
 - ▶ Preemptive: Linux 2.6 Kernel

read system call

